Szoftvertesztelés Projekt Dokumentáció

Python és Pytest alapú egységtesztelés és Architektúra

Lőrincz Emil

2026. Március

Tartalomjegyzék

1. Bevezetés **3**

2. A szoftvertesztelés elméleti alapjai **3**

3. Fejlesztői környezet és a projekt architektúrája **3**

4. A tesztelés megvalósítása **4**

4.1. Alapozó tesztek: A számológép 4

4.2. Üzleti logika és az AAA minta 4

4.3. Negatív tesztelés (Hibakezelés) 5

5. Futtatási eredmények (Kimeneti napló) **5**

6. Összegzés és tapasztalatok **6**

1. Bevezetés

A dokumentáció célja a féléves szoftvertesztelés projekt bemutatása. A feladat során egy meglévő, Python nyelven írt alkalmazotti nyilvántartó rendszert, valamint egy alapvető számológép modult kellett megértenem és automatizált tesztekkel lefednem. A projekt fókuszában az egységtesztek (Unit Tests) álltak, amelyek biztosítják a kód hosszú távú fenntarthatóságát és hibamentességét.

2. A szoftvertesztelés elméleti alapjai

A szoftvertesztelés az a folyamat, amely során ellenőrizzük, hogy egy program a specifikációknak megfelelően működik-e. Az egységtesztelés során a szoftver legkisebb, önmagában is működőképes egységeit teszteljük, elszigetelve a rendszer többi részétől.

Egy jó egységteszt megfelel a **FIRST** elveknek:

- **Fast (Gyors):** A tesztek másodpercek töredéke alatt lefutnak. A jelen projektben a 14 teszteset mindössze 0.09 másodpercet vett igénybe.
- **Isolated (Izolált):** Minden teszt független. Egyik teszt eredménye sem befolyásolja a másikat.
- **Repeatable (Megismételhető):** Bármikor, bármilyen környezetben futtatva ugyanazt a kimenetet kapjuk.
- **Self-validating (Önellenőrző):** Nincs szükség manuális ellenőrzésre, a `pytest` az `assert` parancsok alapján értékeli a futást.

3. Fejlesztői környezet és a projekt architektúrája

A projektet egy izolált virtuális környezetben (`.venv`) futtattam. Ez garantálja, hogy a projekt függőségei (például a `pytest` csomag) nem ütköznek a számítógépen lévő más Python projektekkel. A rendszer logikailag szétbontott modulokból áll:

1. **employee.py:** Az `Employee` osztályt tartalmazza. Ez az adatmodell, amely leírja egy dolgozó tulajdonságait (ID, név, születési dátum, alapbér, belépés dátuma).
2. **relations_manager.py:** Az adatbázist szimuláló réteg. Lekérdezi a dolgozókat, megállapítja a vezetői státuszt (`is_leader`), és visszaadja a csapattagokat (`get_team_members`).
3. **employee_manager.py:** Az üzleti logika helye. Ez a modul végzi a bérszámítást a `calculate_salary` függvénnyel, figyelembe véve a `RelationsManager` által biztosított adatokat.

4. A tesztelés megvalósítása

4.1. Alapozó tesztek: A számológép

A tesztelési keretrendszer megismerését a `calculator.py` tesztelésével kezdtem. Itt az alapműveleteket ellenőriztem.

```python
import pytest
from calculator import add, subtract, multiply

def test_calc_addition():
    assert add(2, 3) == 5

def test_calc_substraction():
    assert subtract(10, 5) == 5

def test_calc_multiply():
    assert multiply(3, 4) == 12
```

4.2. Üzleti logika és az AAA minta

Az alkalmazotti rendszer tesztelését az iparági **AAA (Arrange, Act, Assert)** minta alapján írtam meg. A legösszetettebb feladat a vezetői bérszámítás ellenőrzése volt, amely alapbérből, szolgálati időből és csapattagok után járó bónuszból áll.

```python
def test_9_vezeto_bonusz_szamitas():
    # 1. ARRANGE (Elokeszites)
    rm = RelationsManager()
    em = EmployeeManager(rm)

    john = None
    for e in rm.get_all_employees():
        if e.id == 1:
            john = e  # Megkeressuk John Doe-t, aki vezeto

    # 2. ACT (Vegrehajtas)
    eredmeny = em.calculate_salary(john)

    # 3. ASSERT (Ellenorzes)
    aktualis_ev = datetime.date.today().year
    evek = aktualis_ev - 1990
    # Keplet: Alap(3000) + (Evek * 100) + (2 tag * 200)
    elvart = 3000 + (evek * 100) + (2 * 200)

    assert eredmeny == elvart
```

4.3. Negatív tesztelés (Hibakezelés)

A szoftver robusztusságának igazolására negatív teszteket is írtam. Ezek célja annak ellenőrzése, hogy a rendszer megfelelően, összeomlás nélkül kezeli-e a hibás bemeneteket. A `test_10_nem_letezo_dolgozo_keresese` függvény bizonyítja, hogy egy nem létező dolgozó (pl. "Batman") keresésekor a kód nem dob futásidejű hibát, hanem a biztonságos `None` értékkel tér vissza.

```python
def test_10_nem_letezo_dolgozo_keresese():
    rm = RelationsManager()
    szellem = None
    for e in rm.get_all_employees():
        if e.first_name == "Batman":
            szellem = e
    assert szellem is None
```

5. Futtatási eredmények (Kimeneti napló)

A projekt során összesen 14 egységtesztet hoztam létre. A tesztek futtatását a PyCharm termináljából indítottam. Az alábbi, csonkítatlan napló bizonyítja a szoftver és a tesztek 100%-os sikerességét:

```
C:\Users\Emil\...\python.exe "../_jb_pytest_runner.py"
Testing started at 18:46 ...
Launching pytest with arguments --no-header --no-summary -q

================= test session starts =================
collecting ... collected 14 items

calculator_test.py::test_calc_addition PASSED              [  7%]
calculator_test.py::test_calc_substraction PASSED          [ 14%]
calculator_test.py::test_calc_multiply PASSED              [ 21%]
test_managment.py::test_1_john_doe_adatai PASSED           [ 28%]
test_managment.py::test_2_myrta_torkelson_fizetese PASSED  [ 35%]
test_managment.py::test_3_gretchen_watford_adatai PASSED   [ 42%]
test_managment.py::test_4_john_doe_vezeto_e PASSED         [ 50%]
test_managment.py::test_5_scotty_bomba_nem_vezeto PASSED   [ 57%]
test_managment.py::test_6_john_doe_csapat_merete PASSED    [ 64%]
test_managment.py::test_7_gretchen_csapat_tagjai PASSED    [ 71%]
test_managment.py::test_8_sima_dolgozo_fizetes PASSED      [ 78%]
test_managment.py::test_9_vezeto_bonusz_szamitas PASSED    [ 85%]
test_managment.py::test_10_nem_letezo_dolgozo PASSED       [ 92%]
test_managment.py::test_11_ures_csapat_ha_nem_vezeto PASSED[100%]

================= 14 passed in 0.09s ==================
Process finished with exit code 0
```

6. Összegzés és tapasztalatok

A szoftvertesztelés projekt során gyakorlati tapasztalatot szereztem az automatizált tesztelés terén. Megtanultam, hogyan kell a `pytest` keretrendszert használni, és hogyan kell a kód különböző logikai rétegeit (adatmodell, adatbázis szimuláció, üzleti logika) egymástól függetlenül vizsgálni.

A projekt bizonyította a letöltött forráskód stabilitását. A negatív tesztek és a pontos matematikai validációk garantálják, hogy a rendszer a specifikációban leírtaknak megfelelően működik. A 14 sikeresen lefutott teszt maradéktalanul teljesíti a féléves laborgyakorlat követelményeit.